

P2611R0

2022-07 Library Evolution Poll Outcomes

Published Proposal, 2022-07-27

Authors:

[Bryce Adelstein LeLach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

[Fabio Fracassi — Library Evolution Vice Chair](#) (CODE University of Applied Sciences)

[Ben Craig — Library Evolution Vice Chair](#) (NI)

[Inbal Levi — Ranges Chair \(SG9\) and Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1	Introduction
2	Poll Outcomes
3	Selected Poll Comments
3.1	C++23 Polls
3.1.1	[P0429R9] flat_map
3.1.2	[P1222R4] flat_set
3.1.3	[P0792R10] function_ref
3.1.4	[P2505R4] Monadic Functions For expected
3.1.5	[P2585R0] Improving Default Container Formatting

- 3.1.6 [P2446R2] `views::as_rvalue`
- 3.1.7 [P2278R4] `cbegin` Should Always Return A Constant Iterator
- 3.1.8 [P2248R5] Enabling List-Initialization For Algorithms
- 3.1.9 [P2539R1] Should The Output Of `print` To A Terminal Be Synchronized With The Underlying Stream?
- 3.1.10 [P2510R3] Formatting Pointers
- 3.1.11 [P2551R2] Clarify Intent Of Individually Specializable Numeric Traits
- 3.1.12 [P2599R2] `index_type` ## Poll 12 ## `{#poll-12} size_type` In `mdspan`
- 3.1.13 [P2604R0] `mdspan`: Rename `pointer`, `data`, And `contiguous`
- 3.1.14 [P2613R1]: Add The Missing `empty` To `mdspan`
- 3.2 C++26 and Technical Specification Polls
 - 3.2.1 [P2338R2] Freestanding Library: Character Primitives And The C Library
 - 3.2.2 [P2407R1] Freestanding Library: Partial Classes
 - 3.2.3 [P2562R1] `constexpr` Stable Sorting
 - 3.2.4 [P2283R2] `constexpr` Specialized Memory Algorithms
 - 3.2.5 [P2542R2] `views::concat`
 - 3.2.6 [P2609R1] Relaxing Ranges Just A Smidge

References

Informative References

§ 1. Introduction

In 2022-07, the C++ Library Evolution group conducted a series of electronic decision polls [\[P2610R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 32 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.

- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1.1: Return [P0429R9] flat_map to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	15	13	1	0	0	Strong consensus in favor.
Poll 1.2: Return [P1222R4] flat_set to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	15	13	1	0	0	Strong consensus in favor.
Poll 1.3: Return [P0792R10] function_ref to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	16	7	1	1	1	Consensus in favor.
Poll 1.4: Send [P2505R4] Monadic Functions For expected to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	15	11	0	1	0	Consensus in favor.
Poll 1.5: Send [P2585R0] Improving Default Container Formatting to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	16	7	0	0	1	Consensus in favor.
Poll 1.6: Send [P2446R2] views::as_rvalue to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	15	7	0	1	0	Consensus in favor.
Poll 1.7: Send [P2278R4] cbegin Should Always Return A Constant Iterator to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	16	8	0	0	0	Unanimous consensus in favor.
Poll 1.8: Send [P2248R5] Enabling List-Initialization For Algorithms to Library Working Group for C++23,	6	13	3	1	0	Consensus in favor.

classified as an improvement of an existing feature ([P0592R4] bucket 2 item).						
Poll 1.9: Send [P2539R1] Should The Output Of print To A Terminal Be Synchronized With The Underlying Stream? to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	11	10	2	1	0	Consensus in favor.
Poll 1.10: Send [P2510R3] Formatting Pointers to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	7	12	1	2	1	Poll withdrawn due to incorrect ship vehicle. It will be retaken targeting C++26 in the next electronic polling period.
Poll 1.11: Send [P2551R2] Clarify Intent Of Individually Specializable Numeric Traits to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	12	8	0	0	0	Unanimous consensus in favor.
Poll 1.12: Send [P2599R2] index_type & size_type In mdspan to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	14	7	2	1	0	Consensus in favor.
Poll 1.13: Send [P2604R0] mdspan: Rename pointer, data, And contiguous to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	13	13	0	0	0	Unanimous consensus in favor.
Poll 1.14: Send [P2613R1] : Add The Missing empty To mdspan to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	16	10	1	1	0	Consensus in favor.
Poll 2.1: Send [P2338R2] Freestanding Library: Character	11	6	0	0	0	Unanimous consensus in favor.

Primitives And The C Library to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).						
Poll 2.2: Send [P2407R1] Freestanding Library: Partial Classes to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	7	8	1	0	0	Strong consensus in favor.
Poll 2.3: Send [P2562R1] constexpr Stable Sorting to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	9	7	2	0	0	Strong consensus in favor.
Poll 2.4: Send [P2283R2] constexpr Specialized Memory Algorithms to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	10	7	0	0	0	Unanimous consensus in favor.
Poll 2.5: Send [P2542R2] views::concat to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	10	7	0	0	0	Unanimous consensus in favor.
Poll 2.6: Send [P2609R1] Relaxing Ranges Just A Smidge to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	5	6	2	0	0	Strong consensus in favor.

§ 3. Selected Poll Comments

§ 3.1. C++23 Polls

§ 3.1.1. [\[P0429R9\]](#) `flat_map`

Flat_map and flat_set are generally useful containers and having them in the standard library improve the common vocabulary for developers.

— Strongly Favor

Very useful container adapters. I have written this functionality a lot of times in multiple manners.

— Strongly Favor

You no longer need to write equal_range on bare containers and explain it in the code review. Also, fast enumeration through an ordered map is a very useful characteristic.

— Strongly Favor

The "valid but unspecified state" wording for things like erase_if is too broad in my view. As I understand it, the state is just "some of the elements got erased and some didn't." "Valid but unspecified" is like the after-move wording, which only promises nasal-demon-free execution.

— Weakly Favor

While I'm not thrilled about the timing of the changes, they seem to have been sufficiently reviewed by domain experts.

— Weakly Favor

This is a generally useful vocabulary collection that provides a time/space cost tradeoff that's significantly different to other standard library collections.

— Weakly Favor

§ 3.1.2. [\[P1222R4\]](#) `flat_set`

`Flat_map` and `flat_set` are generally useful containers and having them in the standard library improve the common vocabulary for developers.

— Strongly Favor

While I'm not thrilled about the timing of the changes, they seem to have been sufficiently reviewed by domain experts.

— Weakly Favor

Even though discussions are ongoing on some aspects of this, the proposal seems mature enough to me at this stage. I would use it and so would my customers.

— Weakly Favor

This is a generally useful vocabulary collection that provides a time/space cost tradeoff that's significantly different to other standard library collections.

— Weakly Favor

§ 3.1.3. [\[P0792R10\]](#) `function_ref`

Extremely useful vocabulary type. I regularly need and use `function_ref`-equivalent types in all C++ codebases I work with. I will be disappointed if this does not make it for C++23.

— Strongly Favor

The new constructors provides the following features:

1) Missing functionality by being able to directly handle the most common use cases; stateful free function and stateful member function. It also handles stateless member function making this release more of fix as well. 2) Safer because immediate dangling is removed and general dangling is diminished and made more visible by removing a superfluous functor/stateful lambda 3) Faster in the most common use cases where otherwise a superfluous functor/stateful lambda would have been created 4) Easier to use because the programmer doesn't have to manage the lifetime of a superfluous instance or deal with dangling as much.

— Strongly Favor

Adding new constructors brings an efficient and intuitive solution to address a wide range of demands to bind non-callable objects or migrate Java-style, interface-based code.

— Strongly Favor

Many code bases replicate this vocabulary type, but it has subtle safety pitfalls. I think the authors made the right choices with respect to safety (e.g., Behaviors A.2 and B.2).

— Strongly Favor

The functionality itself is very useful and the design looks good. WF because it does not need to be in C++26.

— Weakly Favor

Still worth keeping, even though we lost consistency when filing off the sharp edges of this facility

— Weakly Favor

I've been deeply concerned about the sharp edges and potential for UB here, in what needs to be as simple to use as `std::function`. I think the latest changes with the nontype constructor finally make it reliable, if a bit less convenient.

— Neutral

I'm not sure LWG has time to review this paper. It is not clear to me that not supporting construction from pmf is the right design (It might be but this paper has undergone a lot of changes recently and that lack of clarity makes me nervous)

— Weakly Against

§ 3.1.4. [\[P2505R4\]](#) Monadic Functions For `expected`

Useful functionality that provides consistency with `optional`. It should be in C++23.

— Strongly Favor

This fixes a significant inconsistency between `std::optional` and `std::expected`, making it much easier to write generic code that accepts either.

— Strongly Favor

I just recently added `std::expected` to my toolbox and immediately felt the lack of monadic functions. I just want it.

— Strongly Favor

Thank you authors for sticking with this.

— Weakly Favor

This seems only natural, and finally puts C++17's tightened-up expression evaluation ordering rules to good use.

— Weakly Favor

I'd be more strongly in favor of this if it were C++26, but I'm not going to stop it from C++23.

— Weakly Favor

there are more important things to do

— Weakly Against

§ 3.1.5. [\[P2585R0\]](#) Improving Default Container Formatting

We need to improve the container formatting before too much code relies on it.

— Strongly Favor

Make the API more user friendly, by giving a right default most often, while not causing any regression. This is API change and should be in C++23.

— Strongly Favor

This is a small but important change that improves container formatting. It would be hard to introduce after C++23 because of its interaction with formatting of ranges.

— Strongly Favor

This seems pretty useful, and certainly gets reinvented a lot. The choices are reasonable. Even though this is getting close to opinionated territory, it's useful enough to be worth standardizing.

— Weakly Favor

Makes range formatting much more useful for getting helpful output for collections of all sorts of types. My only qualm is that `range_format_kind` may not be sufficiently exhaustive, and may be difficult to add to in the future.

— Weakly Favor

The maturity (or lack of it) of the formatting-language creeps me, but not enough to not trust this enough.

— Weakly Favor

I do not think this is necessary in 23, and i do not believe it's the right design. I'd rather see more general "is_associative_container", "is_character_string" type traits that are more generally useful. debug should be entirely separate and orthogonal, etc.

— Strongly Against

§ 3.1.6. [\[P2446R2\]](#) `views::as_rvalue`

With `as_const`, this is a fundamental building view for handling proxy ranges.

— Strongly Favor

Fills a gap in the ranges algorithms that's complicated to get right, but very important in particular in light of the new `ranges::to` materialization operation.

— Strongly Favor

The misguided question of "How do I know whether to move from a range?" comes up quite frequently; it will help to have a standard-library facility to mention in the answer of "You don't; your client uses X to make you do so.".

— Strongly Favor

Fills a niche, but important gap.

— Weakly Favor

Well the name is confusing for ordinary programmers (but `views::move` would be even more confusing and creating inconsistencies). I can live with it, though.

— Weakly Favor

Extending usability of ranges and views is an important addition to C++23

— Weakly Favor

Sounds arcane.

— Weakly Against

§ 3.1.7. [\[P2278R4\]](#) `cbegin` Should Always Return A Constant Iterator

This is a great paper with a lot of analysis. It shows the subtle challenges in solving this apparently simple problem. `views::as_const` is the right name. I appreciate that `const_iterator` tries to be a pointer-to-const if at all possible.

— Strongly Favor

This is a MUST HAVE FIX (without it, I will vote NO to C++23). For sure, it should be a defect against C++20. And of course we also have to fix `std::cbegin()` accordingly as THAT function will also be used by programmers.

— Strongly Favor

Improving `constexpr`-ness and `const` correctness.

— Weakly Favor

This facility is a bit complicated, but it produces a material improvement over all other options (including the status quo).

— Weakly Favor

§ 3.1.8. [\[P2248R5\]](#) Enabling List-Initialization For Algorithms

This looks like a usability win. I'm not completely confident about possible API breakage, but I'm confident that the benefits are worthwhile.

— Strongly Favor

The cognitive overhead of naming the constructed type seems small to me, and I'm not sure I would encourage the style of using list initialization, it seems OK for people who do like this sort of thing.

— Weakly Favor

It could be said that clarity suffers in the context of the long argument lists of many algorithms, but where type information is unambiguously available it is a virtue of expressivity to allow it to be used by the programmer.

— Weakly Favor

I still think that we shouldn't encourage untyped braces being passed around, but the changes are unobjectionable.

— Weakly Favor

Too much work for C++23

— Neutral

Given that we make all non-single argument constructor implicit, I am not convinced that using `std::find(v.begin(), v.end(), {"343", s})` for vector of string is improvement. But that is also matter of code style.

— Neutral

List Initialization just increases the complexity of initialization.

— Weakly Against

§ 3.1.9. **[P2539R1]** Should The Output Of `print` To A Terminal Be Synchronized With The Underlying Stream?

As much as I don't like adding more synchronization, it would be hard to introduce `print` to existing code without this feature.

— Strongly Favor

The only alternative is to require that migration to `std::print` be an all-or-nothing affair, which is absurd.

— Strongly Favor

To quote Davis: "defining writing to a stream via `std::print` as not writing to the stream is broken". This is a correctness issue. Weakly because the wording in the paper doesn't actually realize that, as discovered during the LWG review.

— Weakly Favor

Please standardize existing implementations (in this case: my own). AFAIK the underlying facilities don't require synchronization and gratuitous ones make things worse for no apparent reason.

— Weakly Favor

This design is the result of broad discussion done in the mailing list, the discussion made sure the tradeoffs are clear. And so, I believe this is the right direction.

— Weakly Favor

Lack of implementation experience, and lack of benchmarks in the paper

— Weakly Against

§ 3.1.10. [\[P2510R3\]](#) Formatting Pointers

Thank the authors for the formatting specifications.

— Strongly Favor

A minor but useful change to formatting pointers with `std::format`.

— Weakly Favor

Minor addition, but pointers are an important part of C++ and are broadly used.

— Weakly Favor

This seems fine. It feels like a very low priority enhancement to me, though.

— Neutral

I do not see much utility for this.

— Weakly Against

This doesn't need to go to C++23. The meeting notes say that this has been retargeted to C++26. I think I'm weakly in favor for C++26.

— Strongly Against

§ 3.1.11. [\[P2551R2\]](#) Clarify Intent Of Individually Specializable Numeric Traits

I am very very strongly opposed to standardising new traits that preserve past mistakes with 100% bug compatibility. If you need the old junk, use the old junk. Without this clarification I would argue for my NB to vote against adding the new traits at all. This clarification makes the new traits actually useful.

— Strongly Favor

I keep returning to this when I explain type traits (the practices that evolved over time) and I agree we need some clarifications in these few cases

— Strongly Favor

This is in my mind a clear direction for LWG, and improves the original paper dramatically. We do NOT need to be "bug compatible" with `numeric_limits` with the new traits.

— Strongly Favor

It is useful that these can be specified better than `std::numeric_limits` as well as being more ergonomic to use.

— Weakly Favor

§ 3.1.12. [\[P2599R2\]](#) `index_type` ## Poll 12 ## `{#poll-12} size_type` In `mdspan`

Given the `size_type` debate, this renaming makes a lot of sense.

— Strongly Favor

I see some chance of misuse, but the outcome should be convenient.

— Strongly Favor

Consistency in the connotations and denotations of names in the standard library is important for users.

— Strongly Favor

The reference implementation of `mdspan` already implements this; please don't make the authors change it back.

— Strongly Favor

Makes sense, and it's now or never.

— Weakly Favor

I have been following `mdspan` for a fair bit of time and I especially appreciated the liberation of the index type. I have not had the time to read the latest changes in detail and I am left wondering whether the explicit use of `make_unsigned_t` on the `IndexType` of `mdspan` would restrict the possible types for `IndexType` to only fundamental integral types, since a program is not allowed to add specializations of `make_unsigned`. In other words: could we have used `mdspan` with a custom integer type such as `boost::safe<int>`, which would no longer be possible after we adopt P2599R2? Unfortunately I will not have the time to look into this before the next plenary.

— Neutral

I am not overly concerned with the status quo, because the situation were this occurs is controllable and rare, the fix seems fine.

— Neutral

There was some serious last minute on-the-fly design with this. That's very dangerous when dealing with highly used typedefs like `size_type`.

— Weakly Against

§ 3.1.13. [P2604R0] `mdspan`: Rename `pointer`, `data`, And `contiguous`

We should do this before `mdspan` ships.

— Strongly Favor

These new names will mitigate possibility of misuse of the function in generic context.

— Strongly Favor

The renamings proposed will clearly reduce confusion, even though the possibility of using "surjective" instead of "exhaustive" might be a further improvement.

— Weakly Favor

The rename to `data_handle` is fine. I would rather prefer surjective to exhaustive though instead of contiguous, but it is no reason to block this change.

— Weakly Favor

Not truly necessary since `xtensor` & `pybind11/numpy` simply use `data()` (and `mutable_data()` for `pybind11`). But it's a good change.

— Weakly Favor

§ 3.1.14. [\[P2613R1\]](#): Add The Missing `empty` To `mdspan`

In this case, the `empty` function is required, as the `size() == 0` may not work for the views with non-unique mappings, that have a size that overflow its `size_type`, despite the index never doing so.

— Strongly Favor

Design consistency is an important feature of the STL, and this brings `mdspan` in line with other parts of the library.

— Strongly Favor

Fixes an oversight and increases consistency with `span`.

— Strongly Favor

Yes, this is consistent, and the absence of `empty` would be surprising.

— Weakly Favor

My field experience is that "empty" describes a container of uninitialized values. It doesn't fit here.

— Weakly Against

§ 3.2. C++26 and Technical Specification Polls

§ 3.2.1. [\[P2338R2\]](#) Freestanding Library: Character Primitives And The C Library

This paper proposes well-understood changes that expand the scope of freestanding.

— Strongly Favor

I agree with this direction. The freestanding efforts are important but the less we disallow, the better (let's help our embedded developers if we can)

— Strongly Favor

Cleaning up and creating a consistent and usable freestanding subset is a very useful goal.

— Strongly Favor

Finalizing the work on Freestanding is valuable

— Weakly Favor

§ 3.2.2. [\[P2407R1\]](#) Freestanding Library: Partial Classes

This expands the scope of freestanding and has implementation experience.

— Strongly Favor

Cleaning up and creating a consistent and usable freestanding subset is a very useful goal.

— Strongly Favor

I agree with this direction. The freestanding efforts are important but the less we disallow, the better (let's help our embedded developers if we can)

— Weakly Favor

I'm not thrilled by having optional methods in a class depending on some compiler mode (which this effectively is), but I suppose it is a useful compromise.

— Neutral

§ 3.2.3. [\[P2562R1\]](#) `constexpr` Stable Sorting

If we can make things `constexpr` it is useful to do so, increasing the capability of compile time programming.

— Strongly Favor

Our facilities should be `constexpr` unless there is a strong reason that they should not be.

— Strongly Favor

The sorting is handy, and desired in compile time evaluation. The paper has implementation experience, mitigating significant concerns.

— Strongly Favor

The more the merrier.

— Weakly Favor

More `constexpr` features is important to further extend the strengths of C++.

— Weakly Favor

I'm not sure there is much motivation beyond "why not?" in this paper

— Neutral

§ 3.2.4. [\[P2283R2\]](#) `constexpr` Specialized Memory Algorithms

Exposing standard library machinery that all standard library implementors have already implemented will help users and ensure they are not tempted to use the unspecified parts of their standard library.

— Strongly Favor

Should just have been done in 20.

— Strongly Favor

Constexpr all the things!

— Strongly Favor

More constexpr features is important to further extend the strengths of C++. Since this deals with non-trivial to constexpr things, the paper having implementation experience is reassuring.

— Weakly Favor

§ 3.2.5. [\[P2542R2\]](#) `views::concat`

This is an important ranges algorithm that I wish we would have had in C++20.

— Strongly Favor

The ability to flatten heterogeneous but otherwise compatible ranges into a single range is very useful, especially since views often have subtly different types depending on the exact sequence of operations used to produce them.

— Strongly Favor

`views::concat` has been implemented in range-v3 for long time and it is proven to be very useful.

— Strongly Favor

The more the merrier.

— Weakly Favor

More ranges is better ranges.

— Weakly Favor

§ 3.2.6. [\[P2609R1\]](#) Relaxing Ranges Just A Smidge

This is a minimal fix for a rare but real problem when using projection functions.

— Strongly Favor

This closes an unexpected usage error in ranges

— Strongly Favor

This is a reasonable fix for algorithms that aren't constrained properly for projections

— Strongly Favor

This fixes the gap, that even experts may be unaware of. Wf because of the possible impact on concept subsumption (need to be clarified).

— Weakly Favor

§ References

§ Informative References

[P0429R9]

Zach Laine. *A Standard flat_map*. 17 June 2022. URL: <https://wg21.link/p0429r9>

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P0792R10]

Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo. *[function_ref: a non-owning reference to a Callable](https://wg21.link/p0792r10)*. 14 June 2022. URL: <https://wg21.link/p0792r10>

[P1222R4]

Zach Laine. *[A Standard flat_set](https://wg21.link/p1222r4)*. 13 June 2022. URL: <https://wg21.link/p1222r4>

[P2248R5]

Giuseppe D'Angelo. *[Enabling list-initialization for algorithms](https://wg21.link/p2248r5)*. URL: <https://wg21.link/p2248r5>

[P2278R4]

Barry Revzin. *[cbegin should always return a constant iterator](https://wg21.link/p2278r4)*. 17 June 2022. URL: <https://wg21.link/p2278r4>

[P2283R2]

Michael Schellenberger Costa. *[constexpr for specialized memory algorithms](https://wg21.link/p2283r2)*. 26 November 2021. URL: <https://wg21.link/p2283r2>

[P2338R2]

Ben Craig. *[Freestanding Library: Character primitives and the C library](https://wg21.link/p2338r2)*. 11 November 2021. URL: <https://wg21.link/p2338r2>

[P2407R1]

Emil Meissner, Ben Craig. *[Freestanding Library: Partial Classes](https://wg21.link/p2407r1)*. 23 November 2021. URL: <https://wg21.link/p2407r1>

[P2446R2]

Barry Revzin. *[views::as_rvalue](https://wg21.link/p2446r2)*. 15 February 2022. URL: <https://wg21.link/p2446r2>

[P2505R4]

Jeff Garland. *[Monadic Functions for std::expected](https://wg21.link/p2505r4)*. 17 June 2022. URL: <https://wg21.link/p2505r4>

[P2510R3]

Mark de Wever. *[Formatting pointers](https://wg21.link/p2510r3)*. 23 May 2022. URL: <https://wg21.link/p2510r3>

[P2539R1]

Victor Zverovich. *[Should the output of std::print to a terminal be synchronized with the underlying stream?](https://wg21.link/p2539r1)*. 11 April 2022. URL: <https://wg21.link/p2539r1>

[P2542R2]

Hui Xie, S. Levent Yilmaz. *[views::concat](https://wg21.link/p2542r2)*. 11 May 2022. URL: <https://wg21.link/p2542r2>

[P2551R2]

Matthias Kretz, Jonathan Wakely. *[Clarify intent of P1841 numeric traits](https://wg21.link/p2551r2)*. 22 June 2022. URL: <https://wg21.link/p2551r2>

[P2562R1]

Oliver Rosten. *[constexpr Stable Sorting](https://wg21.link/p2562r1)*. 14 June 2022. URL: <https://wg21.link/p2562r1>

[P2585R0]

Barry Revzin. *Improving default container formatting*. 15 May 2022. URL: <https://wg21.link/p2585r0>

[P2599R2]

Nevin Liber. *index_type & size_type in mdspan*. 23 June 2022. URL: <https://wg21.link/p2599r2>

[P2604R0]

Christian Trott. *MDSPAN: rename pointer and contiguous*. 15 June 2022. URL: <https://wg21.link/p2604r0>

[P2609R1]

John Eivind Helset. *Relaxing Ranges Just A Smidge*. URL: <https://wg21.link/p2609r1>

[P2610R0]

Bryce Adelstein LeBach. *2022-07 Library Evolution Polls*. URL: <https://wg21.link/p2610r0>

[P2613R1]

Yihe Le. *Add the missing `empty` to `mdspan`*. URL: <https://wg21.link/p2613r1>